IOCTL in Kernel Programming

This is a copy of a shared ChatGPT conversation

Below are **detailed notes + working example** of `ioctl()` in Linux kernel (character driver) with **API explanation, kernel module code, and user-space application**.

---

1. What is `ioctl()`?

`ioctl` = **Input/Output Control**

It is a **system call used for device-specific operations** that are not covered by standard system calls like `read()`, `write()`, `open()`, `close()`.

Why ioctl is needed?

Standard syscalls cannot:

- Configure hardware registers
- Change device modes
- Send custom commands to drivers
- Perform control operations (reset, set baud rate, etc.)

So kernel exposes a **command-based interface via ioctl**.

---

2. ioctl System Call Prototype

User-space API:

```c
#include <sys/ioctl.h>

int ioctl(int fd, unsigned long request, ...);
```

Parameters:

Parameter	Meaning
fd	File descriptor of device
request	Command code (IOCTL command)
...	Optional argument (pointer/values)

---

3. Kernel-side ioctl Handler

Inside driver:

```c
long (*unlocked_ioctl)(struct file *file, unsigned int cmd, unsigned long arg);
```

Arguments:

Parameter	Meaning
file	Device file pointer
cmd	IOCTL command
arg	User data (pointer/value)

---

4. IOCTL Command Encoding (Important Concept)

Linux uses macros to define commands:

Macros:

```c
_IO(type, nr)        // no argument
_IOR(type, nr, data) // read from kernel
_IOW(type, nr, data) // write to kernel
_IOWR(type, nr, data)// both read/write
```

Example:

```c
#define MY_MAGIC 'k'

#define IOCTL_SET_VALUE _IOW(MY_MAGIC, 1, int)
#define IOCTL_GET_VALUE _IOR(MY_MAGIC, 2, int)
#define IOCTL_RESET     _IO(MY_MAGIC, 3)
```

5. Full Example: Character Driver with ioctl

(A) Kernel Module Code

ioctl_driver.c

```c
#include <linux/module.h>
#include <linux/fs.h>
#include <linux/uaccess.h>
#include <linux/cdev.h>

#define DEVICE_NAME "myioctldev"
#define MY_MAGIC 'k'

#define IOCTL_SET_VALUE _IOW(MY_MAGIC, 1, int)
#define IOCTL_GET_VALUE _IOR(MY_MAGIC, 2, int)
#define IOCTL_RESET     _IO(MY_MAGIC, 3)
```

```
static int major;
static int stored_value = 0;

static long my_ioctl(struct file *file, unsigned int cmd, unsigned long arg)
{
    int value;

    switch (cmd) {

        case IOCTL_SET_VALUE:
            if (copy_from_user(&value, (int __user *)arg, sizeof(value)))
                return -EFAULT;

            stored_value = value;
            printk(KERN_INFO "IOCTL: Set value = %d\n", stored_value);
            break;

        case IOCTL_GET_VALUE:
            value = stored_value;

            if (copy_to_user((int __user *)arg, &value, sizeof(value)))
                return -EFAULT;

            printk(KERN_INFO "IOCTL: Get value = %d\n", stored_value);
            break;

        case IOCTL_RESET:
            stored_value = 0;
            printk(KERN_INFO "IOCTL: Reset done\n");
            break;

        default:
            return -EINVAL;
    }

    return 0;
}

static int my_open(struct inode *inode, struct file *file)
{
    printk(KERN_INFO "Device opened\n");
    return 0;
}

static int my_close(struct inode *inode, struct file *file)
{
    printk(KERN_INFO "Device closed\n");
    return 0;
}

static struct file_operations fops = {
    .owner = THIS_MODULE,
    .open = my_open,
    .release = my_close,
    .unlocked_ioctl = my_ioctl,
};

static int __init ioctl_init(void)
{
    major = register_chrdev(0, DEVICE_NAME, &fops);
    printk(KERN_INFO "IOCTL Driver loaded, Major: %d\n", major);
    return 0;
}

static void __exit ioctl_exit(void)
{
    unregister_chrdev(major, DEVICE_NAME);
    printk(KERN_INFO "IOCTL Driver unloaded\n");
}

module_init(ioctl_init);
module_exit(ioctl_exit);

MODULE_LICENSE("GPL");
```

(B) Makefile

```
Makefile
obj-m += ioctl_driver.o

all:
	make -C /lib/modules/$(shell uname -r)/build M=$(PWD) modules

clean:
	make -C /lib/modules/$(shell uname -r)/build M=$(PWD) clean
```

6. User Space Application

user_app.c

```
C
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/ioctl.h>

#define MY_MAGIC 'k'

#define IOCTL_SET_VALUE _IOW(MY_MAGIC, 1, int)
#define IOCTL_GET_VALUE _IOR(MY_MAGIC, 2, int)
#define IOCTL_RESET     _IO(MY_MAGIC, 3)

int main()
{
    int fd;
    int value = 42;
    int read_value = 0;

    fd = open("/dev/myioctldev", O_RDWR);
    if (fd < 0) {
        perror("Open failed");
        return -1;
    }

    // Set value
    ioctl(fd, IOCTL_SET_VALUE, &value);
    printf("Sent value: %d\n", value);

    // Get value
    ioctl(fd, IOCTL_GET_VALUE, &read_value);
    printf("Received value: %d\n", read_value);

    // Reset
    ioctl(fd, IOCTL_RESET);

    close(fd);
    return 0;
}
```

7. How ioctl Works Internally

Flow:

```
User App
   |
   | ioctl(fd, cmd, arg)
```

```
   v
Kernel System Call Layer
   |
   v
VFS (Virtual File System)
   |
   v
Driver's unlocked_ioctl()
   |
   v
Hardware / internal driver logic
```

8. Important Kernel APIs Used

1. copy_from_user()

```c
copy_from_user(kernel_ptr, user_ptr, size);
```

- Copies data from user → kernel safely

---

2. copy_to_user()

```c
copy_to_user(user_ptr, kernel_ptr, size);
```

- Copies data from kernel → user

---

3. printk()

Kernel logging:

```c
printk(KERN_INFO "message\n");
```

9. Why ioctl Uses “Magic Number”?

Example:

```c
#define MY_MAGIC 'k'
```

Purpose:

- Avoid conflicts between drivers
- Each driver gets unique command namespace
- Ensures kernel can validate ioctl commands

---

10. Advantages of ioctl

- Flexible device control interface
- Supports complex operations
- Efficient (single syscall for many commands)
- Widely used in real drivers (serial, video, network)

---

11. Disadvantages

- Not standardized across devices
- Hard to maintain large ioctl sets
- Poor type safety (compared to modern interfaces like netlink/sysfs)

---

12. Real-world Use Cases

- Serial port configuration (baud rate, parity)
- Network interface control (IP settings)
- Disk operations
- Camera / V4L2 controls
- GPIO control in embedded Linux

---

13. Interview Key Points (Important)

- ioctl is used for **device-specific commands**
- Implemented via **`unlocked_ioctl`** in kernel
- Uses **`copy_from_user`** / **`copy_to_user`**
- Commands created using **`_IO`**, **`_IOR`**, **`_IOW`**
- Requires **magic number + command number**

If you want, I can also provide:

- 🔥 ioctl debugging techniques (dmesg + strace)
- 🔥 modern replacement comparison (sysfs vs ioctl vs netlink)
- 🔥 GPIO ioctl driver example (embedded interview favorite)
- 🔥 kernel 6.x updated ioctl APIs

Just tell me 👍

> Show me a GPIO ioctl driver example (embedded interview favorite)

Below is a **classic embedded Linux interview-grade GPIO ioctl driver example** using a **character device + ioctl interface** to control GPIO pins.

This is a simplified but realistic model of how older/low-level embedded drivers expose GPIO control.

---

1. What this driver will do

We will implement:

IOCTL commands:

- Export GPIO (simulate)
- Set direction (IN/OUT)
- Write GPIO value (0/1)
- Read GPIO value

Architecture:

```
User App
   ↓ ioctl()
GPIO Char Driver
   ↓
GPIO register / sysfs / mock hardware
```

For interview clarity, we simulate GPIO using a **software variable**, but structure is identical to real drivers.

---

2. IOCTL Command Definitions

```c
#define GPIO_MAGIC 'G'

#define GPIO_EXPORT      _IOW(GPIO_MAGIC, 1, int)
#define GPIO_SET_DIR     _IOW(GPIO_MAGIC, 2, int)
#define GPIO_WRITE       _IOW(GPIO_MAGIC, 3, int)
#define GPIO_READ        _IOR(GPIO_MAGIC, 4, int)
```

3. Kernel Driver (GPIO ioctl char driver)

`gpio_ioctl_driver.c`

```c
#include <linux/module.h>
#include <linux/fs.h>
#include <linux/uaccess.h>

#define DEVICE_NAME "gpioioctl"
#define GPIO_MAGIC 'G'

#define GPIO_EXPORT      _IOW(GPIO_MAGIC, 1, int)
#define GPIO_SET_DIR     _IOW(GPIO_MAGIC, 2, int)
```

```c
#define GPIO_WRITE       _IOW(GPIO_MAGIC, 3, int)
#define GPIO_READ        _IOR(GPIO_MAGIC, 4, int)

static int major;

/* Simulated GPIO state */
static int gpio_exported = 0;
static int gpio_direction = 0; // 0=input, 1=output
static int gpio_value = 0;

struct gpio_data {
    int pin;
    int value;
};

/* IOCTL handler */
static long gpio_ioctl(struct file *file, unsigned int cmd, unsigned long arg)
{
    struct gpio_data data;

    switch (cmd) {

    case GPIO_EXPORT:
        if (copy_from_user(&data, (void __user *)arg, sizeof(data)))
            return -EFAULT;

        gpio_exported = 1;
        printk(KERN_INFO "GPIO: Exported pin %d\n", data.pin);
        break;

    case GPIO_SET_DIR:
        if (copy_from_user(&data, (void __user *)arg, sizeof(data)))
            return -EFAULT;

        gpio_direction = data.value;
        printk(KERN_INFO "GPIO: Direction set to %s\n",
               gpio_direction ? "OUTPUT" : "INPUT");
        break;

    case GPIO_WRITE:
        if (copy_from_user(&data, (void __user *)arg, sizeof(data)))
            return -EFAULT;

        if (!gpio_exported || gpio_direction != 1) {
            printk(KERN_WARNING "GPIO: Write not allowed\n");
            return -EPERM;
        }

        gpio_value = data.value;
        printk(KERN_INFO "GPIO: Value written = %d\n", gpio_value);
        break;

    case GPIO_READ:
        if (!gpio_exported)
            return -EPERM;

        data.value = gpio_value;

        if (copy_to_user((void __user *)arg, &data, sizeof(data)))
            return -EFAULT;

        printk(KERN_INFO "GPIO: Value read = %d\n", gpio_value);
        break;

    default:
        return -EINVAL;
    }

    return 0;
}

/* file ops */
static struct file_operations fops = {
    .owner = THIS_MODULE,
    .unlocked_ioctl = gpio_ioctl,
```

```
};

/* init */
static int __init gpio_init(void)
{
    major = register_chrdev(0, DEVICE_NAME, &fops);
    printk(KERN_INFO "GPIO ioctl driver loaded. Major=%d\n", major);
    return 0;
}

/* exit */
static void __exit gpio_exit(void)
{
    unregister_chrdev(major, DEVICE_NAME);
    printk(KERN_INFO "GPIO ioctl driver unloaded\n");
}

module_init(gpio_init);
module_exit(gpio_exit);

MODULE_LICENSE("GPL");
```

4. User Space Application

gpio_user_app.c

```
C
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/ioctl.h>

#define GPIO_MAGIC 'G'

#define GPIO_EXPORT      _IOW(GPIO_MAGIC, 1, int)
#define GPIO_SET_DIR     _IOW(GPIO_MAGIC, 2, int)
#define GPIO_WRITE       _IOW(GPIO_MAGIC, 3, int)
#define GPIO_READ        _IOR(GPIO_MAGIC, 4, int)

struct gpio_data {
    int pin;
    int value;
};

int main()
{
    int fd;
    struct gpio_data data;
    struct gpio_data read_back;

    fd = open("/dev/gpioioctl", O_RDWR);
    if (fd < 0) {
        perror("open failed");
        return -1;
    }

    /* Export GPIO 17 */
    data.pin = 17;
    ioctl(fd, GPIO_EXPORT, &data);

    /* Set direction OUTPUT */
    data.value = 1;
    ioctl(fd, GPIO_SET_DIR, &data);

    /* Write GPIO HIGH */
    data.value = 1;
    ioctl(fd, GPIO_WRITE, &data);
    printf("GPIO set HIGH\n");
```

```
    /* Read GPIO */
    ioctl(fd, GPIO_READ, &read_back);
    printf("GPIO read value = %d\n", read_back.value);

    /* Write GPIO LOW */
    data.value = 0;
    ioctl(fd, GPIO_WRITE, &data);
    printf("GPIO set LOW\n");

    ioctl(fd, GPIO_READ, &read_back);
    printf("GPIO read value = %d\n", read_back.value);

    close(fd);
    return 0;
}
```

5. How this maps to real hardware GPIO drivers

In real embedded Linux:

Instead of:

```
C
gpio_value = data.value;
```

You would do:

```
C
gpio_set_value(pin, value);
```

or modern kernel:

```
C
gpiod_set_value(desc, value);
```

6. Real Kernel APIs used in production GPIO drivers

1. GPIO descriptor API (modern Linux)

```
C
struct gpio_desc *desc;

desc = gpiod_get(dev, "enable", GPIOD_OUT_LOW);
gpiod_set_value(desc, 1);
```

2. Legacy GPIO API

```
C
gpio_request(pin, "mygpio");
gpio_direction_output(pin, 1);
gpio_set_value(pin, 1);
```

7. Interview Talking Points (VERY IMPORTANT)

Why ioctl for GPIO?

- Flexible low-level control
- Used in older BSPs (before sysfs/gpiod)
- Allows custom hardware commands

Why is ioctl replaced today?

- sysfs (deprecated)
- gpiod (preferred modern API)
- character ioctl is not scalable

What interviewer expects:

You should say:

> "GPIO ioctl drivers are legacy but still used in custom embedded BSPs where vendor-specific hardware control is required."

---

8. Common Interview Follow-ups

Q1: What happens if multiple processes access GPIO ioctl?

- Race conditions → need locking (mutex/spinlock)

Q2: How to make it thread-safe?

- Use:

```
C
mutex_lock(&gpio_lock);
```

Q3: Why copy_from_user is needed?

- User space memory cannot be accessed directly in kernel

---

9. Enhancement (Advanced Idea)

You can extend this driver with:

- Interrupt-based GPIO (IRQ ioctl)
- Debounce handling
- PWM control via ioctl
- GPIO direction locking

If you want next level:

I can also give you:

🔥 GPIO IRQ interrupt driver with ioctl + poll()
🔥 Real Raspberry Pi GPIO kernel module
🔥 GPIO using gpiod (modern kernel 5/6 style)
🔥 Interview Q&A with tricky GPIO scenarios

Just tell 👍

> Give me a GPIO IRQ interrupt driver with ioctl + poll()

Below is a **real embedded Linux–style GPIO IRQ character driver** that supports:

- ✅ GPIO interrupt handling (IRQ)
- ✅ `ioctl()` control interface
- ✅ `poll()` / `select()` support for user-space waiting
- ✅ Blocking + event notification design (very interview-relevant)

This is the kind of design used in **button drivers, sensor interrupts, wake-up pins, etc.**

---

1. What this driver does

We simulate a GPIO interrupt pin (like a button):

Features:

- Configure GPIO (simulated)
- Enable interrupt (rising/falling edge)
- On interrupt → wake up user-space
- User-space can:
  - `poll()` to wait for event
  - `ioctl()` to control driver

---

2. IOCTL Interface

```c
#define GPIO_MAGIC 'G'

#define GPIO_SET_PIN     _IOW(GPIO_MAGIC, 1, int)
#define GPIO_SET_EDGE    _IOW(GPIO_MAGIC, 2, int)
#define GPIO_ENABLE_IRQ  _IO(GPIO_MAGIC, 3)
#define GPIO_DISABLE_IRQ _IO(GPIO_MAGIC, 4)
#define GPIO_GET_EVENT   _IOR(GPIO_MAGIC, 5, int)
```

3. Concept of IRQ + poll()

Core idea:

When interrupt happens:

```
IRQ Handler → store event → wake_up_interruptible()
```

User-space:

```
poll() waits on waitqueue → wakes when IRQ occurs
```

4. Kernel Driver (IRQ + ioctl + poll)

gpio_irq_ioctl_driver.c

```c
C
#include <linux/module.h>
#include <linux/fs.h>
#include <linux/uaccess.h>
#include <linux/interrupt.h>
#include <linux/poll.h>
#include <linux/wait.h>

#define DEVICE_NAME "gpioirqdev"
#define GPIO_MAGIC 'G'

#define GPIO_SET_PIN     _IOW(GPIO_MAGIC, 1, int)
#define GPIO_SET_EDGE    _IOW(GPIO_MAGIC, 2, int)
#define GPIO_ENABLE_IRQ  _IO(GPIO_MAGIC, 3)
#define GPIO_DISABLE_IRQ _IO(GPIO_MAGIC, 4)
#define GPIO_GET_EVENT   _IOR(GPIO_MAGIC, 5, int)

static int major;

/* Simulated GPIO IRQ state */
static int gpio_pin = 17;
static int irq_enabled = 0;
static int event_flag = 0;
static int irq_count = 0;

/* wait queue for poll/select */
static DECLARE_WAIT_QUEUE_HEAD(gpio_wq);

/* ---------------- IRQ Handler (SIMULATED) ---------------- */
static irqreturn_t gpio_irq_handler(int irq, void *dev_id)
{
    if (!irq_enabled)
        return IRQ_NONE;

    irq_count++;
    event_flag = 1;

    printk(KERN_INFO "GPIO IRQ occurred! count=%d\n", irq_count);

    wake_up_interruptible(&gpio_wq);

    return IRQ_HANDLED;
}

/* --------------- POLL FUNCTION --------------- */
static unsigned int gpio_poll(struct file *file, poll_table *wait)
{
```

```
    unsigned int mask = 0;

    poll_wait(file, &gpio_wq, wait);

    if (event_flag)
        mask |= POLLIN | POLLRDNORM;

    return mask;
}

/* --------------- IOCTL HANDLER --------------- */
static long gpio_ioctl(struct file *file, unsigned int cmd, unsigned long arg)
{
    int val;

    switch (cmd) {

    case GPIO_SET_PIN:
        if (copy_from_user(&val, (int __user *)arg, sizeof(val)))
            return -EFAULT;

        gpio_pin = val;
        printk(KERN_INFO "GPIO: pin set to %d\n", gpio_pin);
        break;

    case GPIO_ENABLE_IRQ:
        irq_enabled = 1;
        event_flag = 0;
        printk(KERN_INFO "GPIO IRQ enabled\n");
        break;

    case GPIO_DISABLE_IRQ:
        irq_enabled = 0;
        printk(KERN_INFO "GPIO IRQ disabled\n");
        break;

    case GPIO_GET_EVENT:
        val = event_flag;

        if (copy_to_user((int __user *)arg, &val, sizeof(val)))
            return -EFAULT;

        event_flag = 0;
        printk(KERN_INFO "GPIO event read\n");
        break;

    default:
        return -EINVAL;
    }

    return 0;
}

/* --------------- FILE OPS --------------- */
static struct file_operations fops = {
    .owner = THIS_MODULE,
    .unlocked_ioctl = gpio_ioctl,
    .poll = gpio_poll,
};

/* --------------- INIT --------------- */
static int __init gpio_init(void)
{
    major = register_chrdev(0, DEVICE_NAME, &fops);

    printk(KERN_INFO "GPIO IRQ ioctl driver loaded. Major=%d\n", major);

    /* NOTE:
     * In real hardware:
     * request_irq(GPIO_IRQ, gpio_irq_handler, ...)
     *
     * Here we simulate IRQ externally or via test trigger.
     */

    return 0;
```

```
}

/* ---------------- EXIT ---------------- */
static void __exit gpio_exit(void)
{
    unregister_chrdev(major, DEVICE_NAME);
    printk(KERN_INFO "GPIO IRQ driver unloaded\n");
}

module_init(gpio_init);
module_exit(gpio_exit);

MODULE_LICENSE("GPL");
```

5. How IRQ is actually wired (real hardware)

In real embedded system:

```
C
request_irq(irq_number,
            gpio_irq_handler,
            IRQF_TRIGGER_RISING,
            "gpio_irq",
            dev);
```

Hardware triggers:

- Button press
- Sensor event
- Edge detection on GPIO pin

---

6. User-space application using ioctl + poll()

gpio_irq_user.c

```
C
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/ioctl.h>
#include <poll.h>

#define GPIO_MAGIC 'G'

#define GPIO_SET_PIN     _IOW(GPIO_MAGIC, 1, int)
#define GPIO_ENABLE_IRQ  _IO(GPIO_MAGIC, 3)
#define GPIO_GET_EVENT   _IOR(GPIO_MAGIC, 5, int)

int main()
{
    int fd;
    int pin = 17;
    int event = 0;

    struct pollfd pfd;

    fd = open("/dev/gpioirqdev", O_RDWR);
    if (fd < 0) {
        perror("open failed");
        return -1;
```

```
    }

    /* Set GPIO pin */
    ioctl(fd, GPIO_SET_PIN, &pin);

    /* Enable IRQ */
    ioctl(fd, GPIO_ENABLE_IRQ);

    pfd.fd = fd;
    pfd.events = POLLIN;

    printf("Waiting for GPIO interrupt...\n");

    while (1) {

        int ret = poll(&pfd, 1, -1); // blocking wait

        if (ret > 0) {

            if (pfd.revents & POLLIN) {

                ioctl(fd, GPIO_GET_EVENT, &event);

                printf("GPIO INTERRUPT OCCURRED! event=%d\n", event);
            }
        }
    }

    close(fd);
    return 0;
}
```

7. Execution Flow (VERY IMPORTANT)

```
User App
   |
   | poll()
   v
Kernel poll waits on waitqueue
   |
IRQ happens (button press / sensor)
   |
gpio_irq_handler()
   |
event_flag = 1
wake_up_interruptible()
   |
poll() returns
   |
ioctl(GET_EVENT)
```

8. Key Kernel Concepts Used

1. waitqueue

```
C
DECLARE_WAIT_QUEUE_HEAD(gpio_wq);
```

Used for blocking processes until event occurs.

---

2. poll_wait()

```
C
```

```
poll_wait(file, &gpio_wq, wait);
```

Registers process into waitqueue.

3. wake_up_interruptible()

```
C
wake_up_interruptible(&gpio_wq);
```

Wakes sleeping processes when IRQ occurs.

4. IRQ handler return types

Return	Meaning
IRQ_HANDLED	IRQ processed
IRQ_NONE	Not handled

9. Real-world usage

This pattern is used in:

- 📱 Button drivers (volume keys, power button)
- 🛰️ Sensor interrupts (IMU, accelerometer)
- 🔔 Door sensors / alarms
- 📶 Modem / network event notifications

10. Interview Key Points

Q1: Why use poll instead of busy loop?

- Saves CPU
- Kernel wakes process only on event

Q2: Difference between IRQ and poll?

- IRQ: hardware event
- poll: user-space waiting mechanism

Q3: Why waitqueue is needed?

- To block processes efficiently

Q4: What happens if IRQ comes before poll()?

- event_flag stores state → poll will still detect it

---

11. Advanced Improvements (what interviewers love)

You can extend this with:

- debounce logic for buttons
- edge selection via ioctl
- multiple GPIO lines
- IRQ sharing (IRQF_SHARED)
- sysfs + ioctl hybrid driver
- epoll support

If you want next level:

I can also give you:

🔥 Real Raspberry Pi GPIO IRQ driver (actual hardware)
🔥 Debounce-safe button driver (production-grade)
🔥 GPIO + character + sysfs hybrid driver
🔥 Kernel interview questions based on this exact code

Just tell 👍